C.E.S.A.R school

TEORIA DOS GRAFOS

Profª Laura Pacifico

2026 | OUTUBRO

Por que surgiu o NoSQL?

C.E.S.A.R school

O modelo relacional

- Dados em tabelas com esquema fixo
- Relações por chaves estrangeiras e JOINs
- Linguagem SQL padronizada
- Transações ACID
- Cresce, em geral, com uma máquina maior

Os 3 Vs da web em grande escala

V **Volume:** Dados demais para um único servidor: é preciso distribuir em muitas máquinas.

V **Variedade:** JSON, logs, sensores, redes: dados que não têm sempre a mesma estrutura.

V **Velocidade:** Milhões de leituras e escritas por segundo, em tempo real.

NoSQL = Not only SQL. Não substitui o relacional: é um conjunto de modelos diferentes para problemas diferentes.

Relacional x NoSQL

	Relacional	NoSQL
Modelo	Tabelas com linhas e colunas	Chave-valor, documento, colunar, grafo
Esquema	Fixo, definido antes dos dados	Flexível, varia entre registros
Escala	Principalmente vertical	Horizontal (mais máquinas)
Consistência	Transações ACID	De ACID à consistência eventual
Consulta	SQL padronizado	Linguagem própria (Cypher, MQL, CQL)
Relações	Chaves estrangeiras e JOINs	Embutidas, por referência ou nativas

Não é uma disputa. Muitos sistemas usam vários bancos ao mesmo tempo, cada um no que faz melhor: é a persistência poliglota.

Os quatro modelos NoSQL



Chave-valor

Valor acessado por uma chave única, como um dicionário.

Exemplos
Redis, Amazon DynamoDB

Usos comuns
cache, sessões, carrinho



Documento

Registros em JSON, cada um com sua estrutura.

Exemplos
MongoDB, CouchDB

Usos comuns
catálogos, perfis, conteúdo

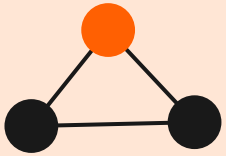


Colunar

Famílias de colunas espalhadas por muitos servidores.

Exemplos
Apache Cassandra, HBase

Usos comuns
IoT, logs, séries temporais



Grafo

Nós e relacionamentos são dados de primeira classe.

Exemplos
Neo4j, Amazon Neptune

Usos comuns
redes sociais, recomendação, rotas

O mesmo dado em quatro modelos

Informação: *Ana cursa Teoria dos Grafos e é amiga de Bruno.*

Chave-valor

```
chave: "aluno:1"
valor: "{nome: Ana,
  disciplina: Teoria dos Grafos,
  amigo: aluno:2}"
```

Documento (JSON)

```
{ "_id": 1,
  "nome": "Ana",
  "disciplinas": ["Teoria dos Grafos"],
  "amigos": [2] }
```

Colunar (linha da tabela)

```
id | nome | disciplina         | amigo
1  | Ana  | Teoria dos Grafos | 2
```

Grafo (Cypher)

```
(ana:Aluno {nome: "Ana"})
  -[:CURSA]->(:Disciplina {nome: "Grafos"}),
(ana)-[:AMIGO_DE]->(:Aluno {nome: "Bruno"})
```

Só no grafo a amizade é um elemento próprio, com tipo, direção e propriedades. Nos outros, é só um identificador guardado no registro.

Por que um banco de grafos?

Relacional: amigos dos amigos da Ana

```sql
SELECT DISTINCT p3.nome
FROM pessoa p1
JOIN amizade a1 ON a1.pessoa_id = p1.id
JOIN amizade a2 ON a2.pessoa_id = a1.amigo_id
JOIN pessoa p3 ON p3.id = a2.amigo_id
WHERE p1.nome = 'Ana';
```

Cada salto a mais é mais um JOIN, calculado na hora da consulta.

Grafo: amigos dos amigos da Ana

```cypher
MATCH (:Pessoa {nome: "Ana"})
      -[:AMIGO_DE*2]->(fof:Pessoa)
RETURN DISTINCT fof.nome;
```

Mais um salto? Troque *2 por *3. A consulta segue as arestas a partir da Ana.

Lembra da lista de adjacência? Em um banco de grafos nativo, cada nó aponta diretamente para seus vizinhos. O custo da consulta depende da vizinhança percorrida, não do tamanho total do banco.

Neo4j em uma página

Neo4j

Banco de dados de grafos nativo: nós e relacionamentos são guardados como estão no modelo, sem tradução para tabelas.

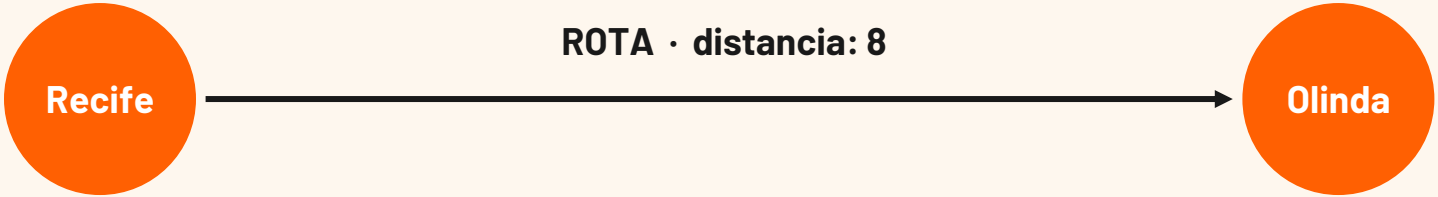
1. **Grafo de propriedades:** Nós, relacionamentos, rótulos e propriedades.
2. **Cypher:** Linguagem declarativa baseada em padrões; inspirou o GQL, padrão ISO publicado em 2024.
3. **Transações ACID:** As mesmas garantias de um banco relacional.
4. **AuraDB:** Versão gerenciada na nuvem, com plano gratuito. É o que vamos usar hoje.

Do grafo matemático ao Neo4j

Teoria dos Grafos	No Neo4j	Exemplo da aula
Vértice $v \in V$	Nó	`(:Cidade {nome: "Recife"})`
Aresta $(u, v) \in E$	Relacionamento	`(recife)-[:ROTA]->(olinda)`
Grafo orientado	Todo relacionamento tem direção	`->`
Peso $w(u, v)$	Propriedade do relacionamento	`{distancia: 8}`
Classe do vértice	Rótulo (label)	`:Cidade`
Atributo do vértice	Propriedade do nó	`nome: "Recife"`

Modelo de grafo de propriedades: nós e relacionamentos guardam pares chave: valor. O Neo4j sempre grava a direção, mas a consulta pode ignorá-la escrevendo (a)-[:ROTA]-(b).

Anatomia de um padrão Cypher




```
(r:Cidade {nome: "Recife"})
```

```
-[:ROTA {distancia: 8}]->
```

```
(o:Cidade {nome: "Olinda"})
```

Nó

Fica entre parênteses ( ). r é a variável, :Cidade é o rótulo e {nome: ...} guarda as propriedades.

Relacionamento

Fica entre colchetes [ ]. :ROTA é o tipo, {distancia: 8} é o peso e a seta -> indica a direção.

Variáveis

r e o só existem dentro do comando. Sem variável o nó fica anônimo: (:Cidade).

Cláusulas essenciais do Cypher

C.E.S.A.R school

CREATE

Cria nós e relacionamentos. Sempre cria, mesmo que já exista.

MERGE

Cria somente se o padrão ainda não existir.

MATCH

Procura um padrão no grafo.

WHERE

Filtra o que o MATCH encontrou.

RETURN

Define o que aparece no resultado.

ORDER BY · LIMIT

Ordena e limita as linhas do resultado.

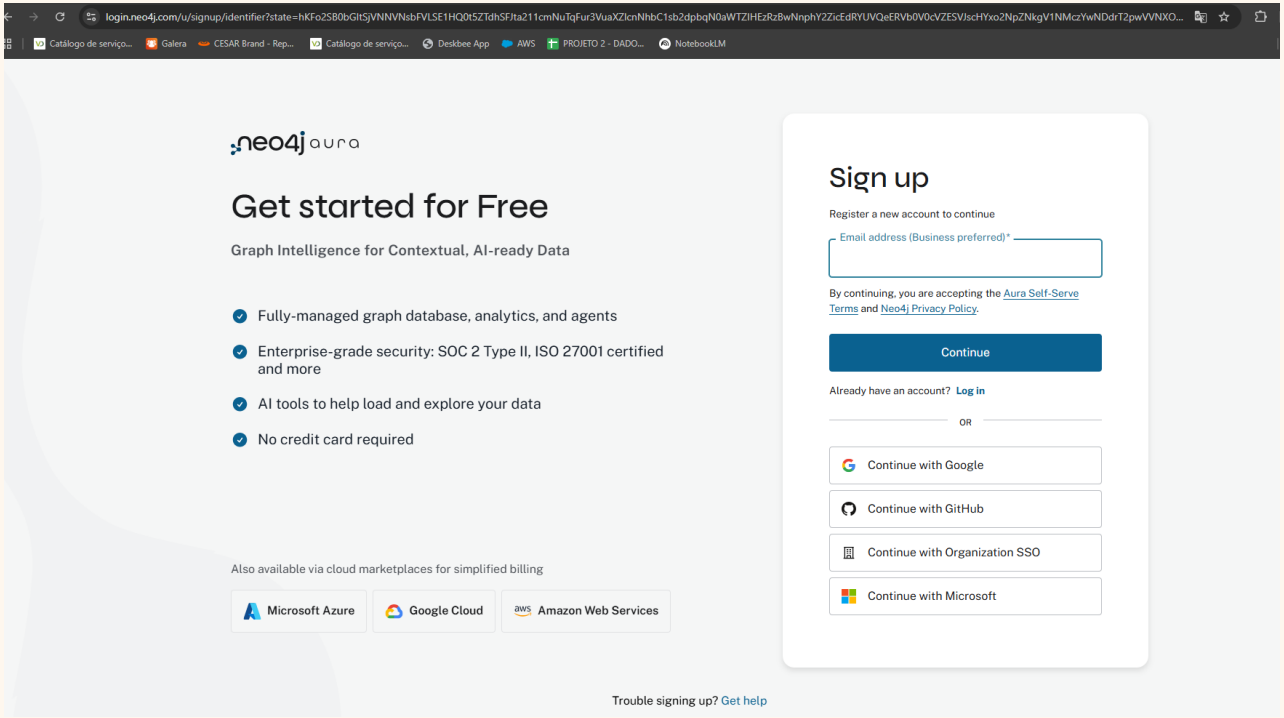
SET

Cria ou altera propriedades.

DETACH DELETE

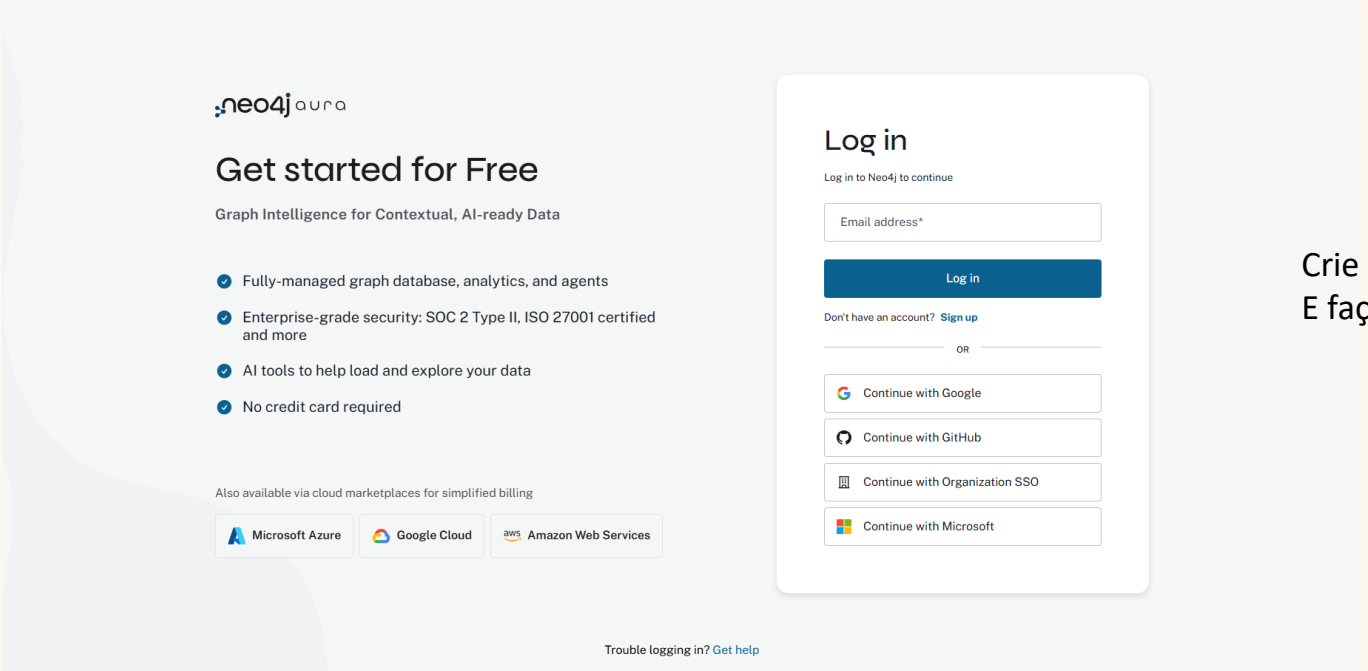
Apaga o nó junto com seus relacionamentos.

Site inicial: neo4j.com



Clicar em “Get Started” e depois em “Neo4j AuraDB (Start Free)”

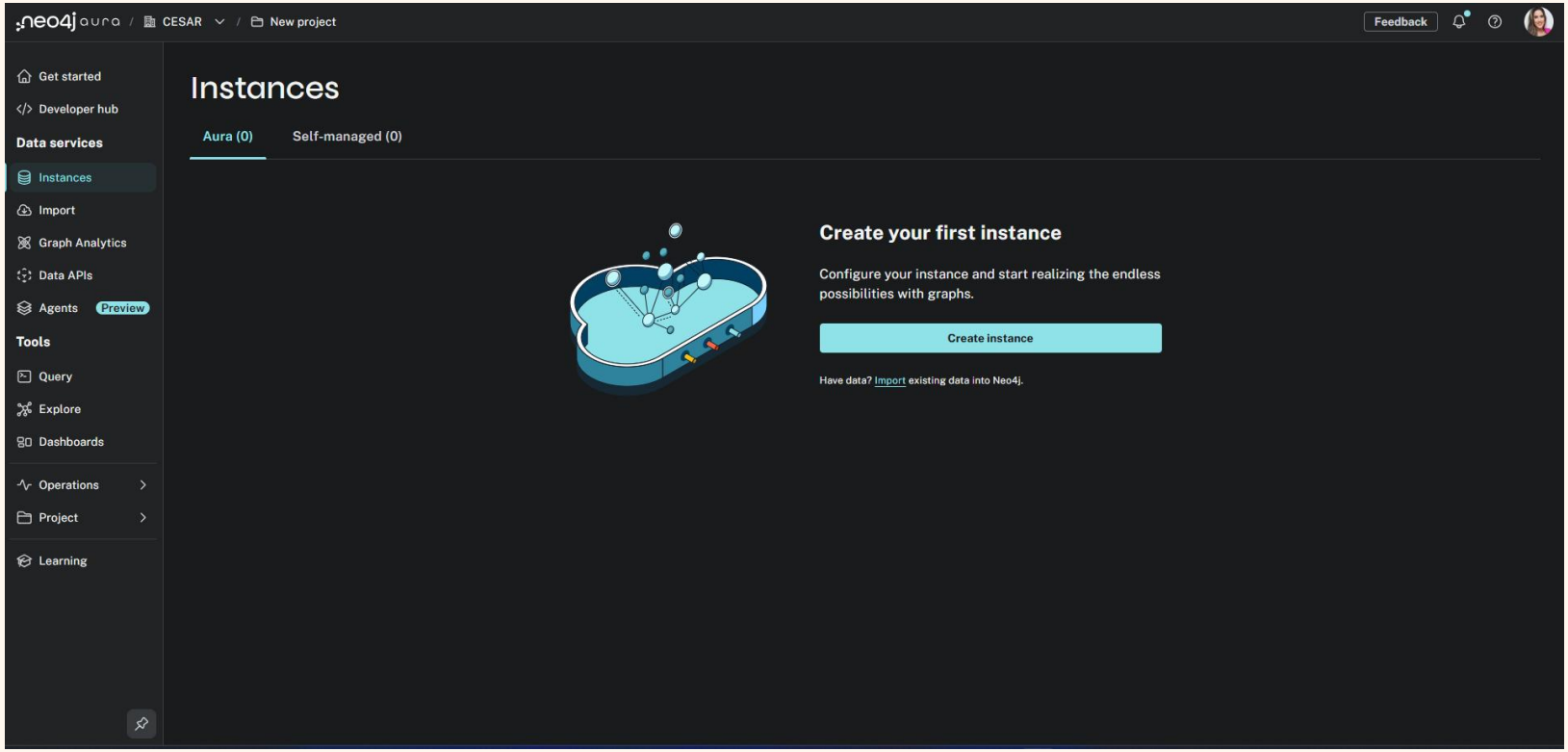
Site inicial: neo4j.com



Crie uma conta gratuita com seu email
E faça o Login.

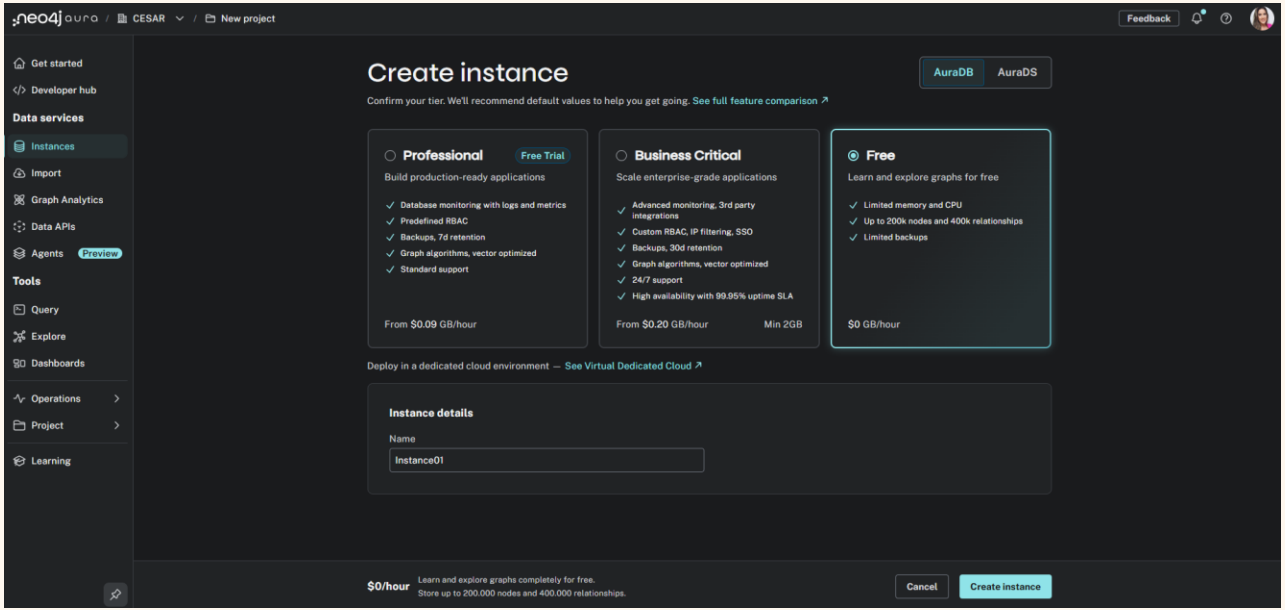
Site inicial: neo4j.com

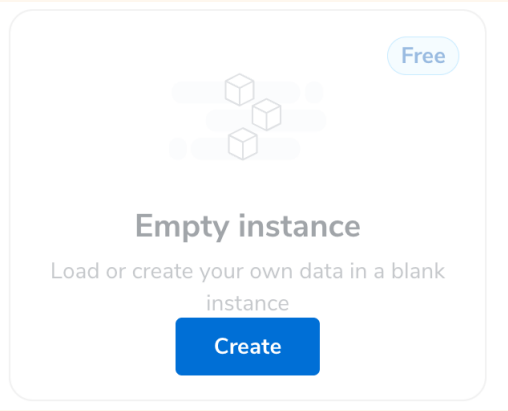
Clicar em “New Instance”



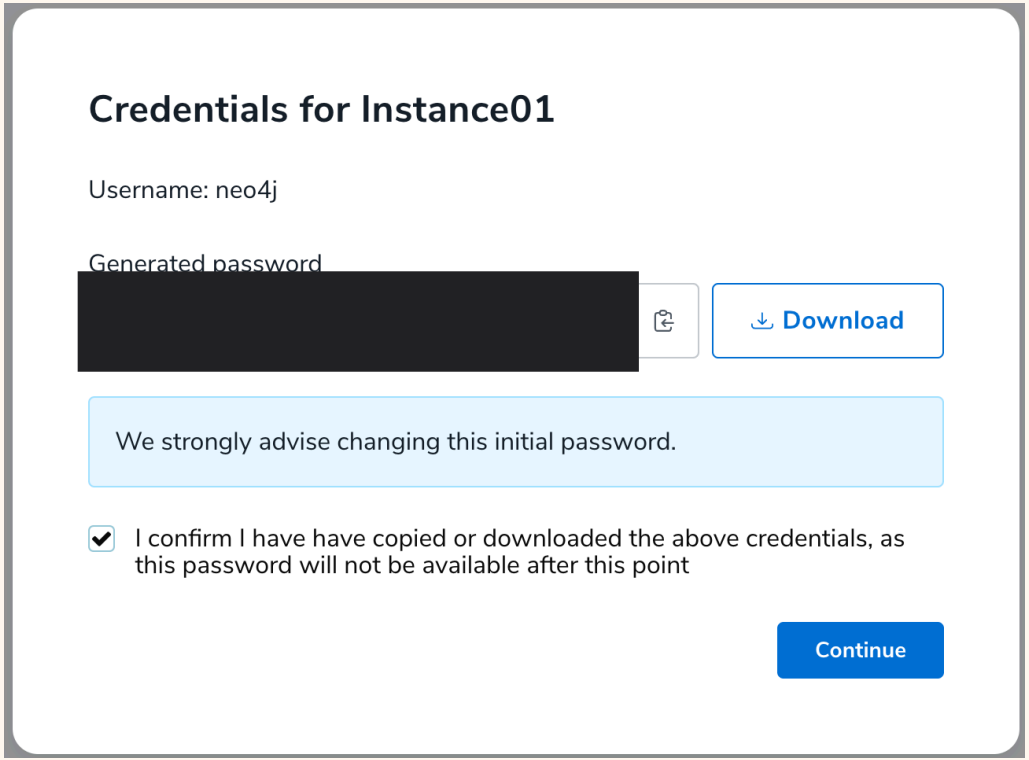
Criando uma instância do Banco AuraDB

Clicar em “Empty instance - Create”



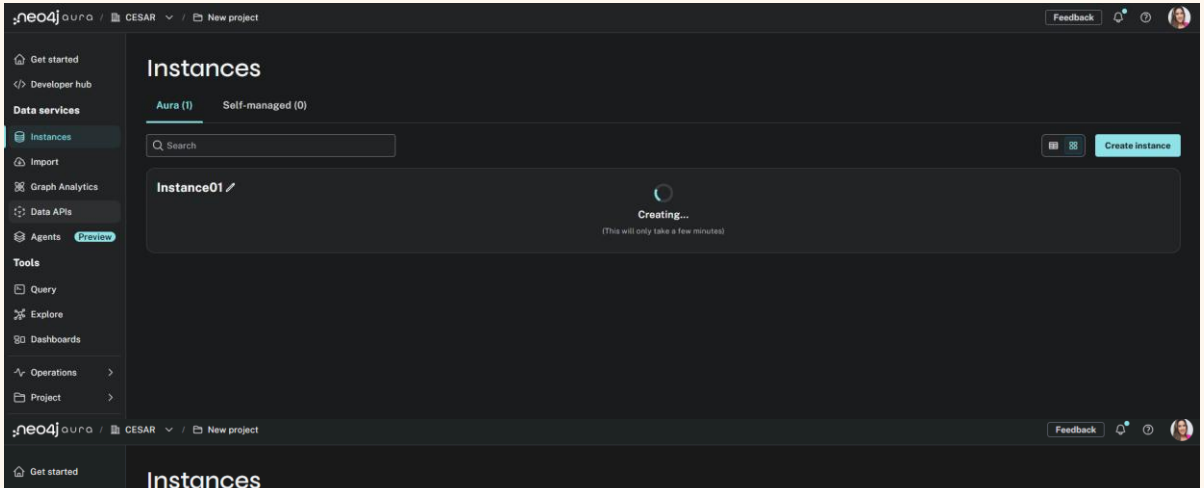


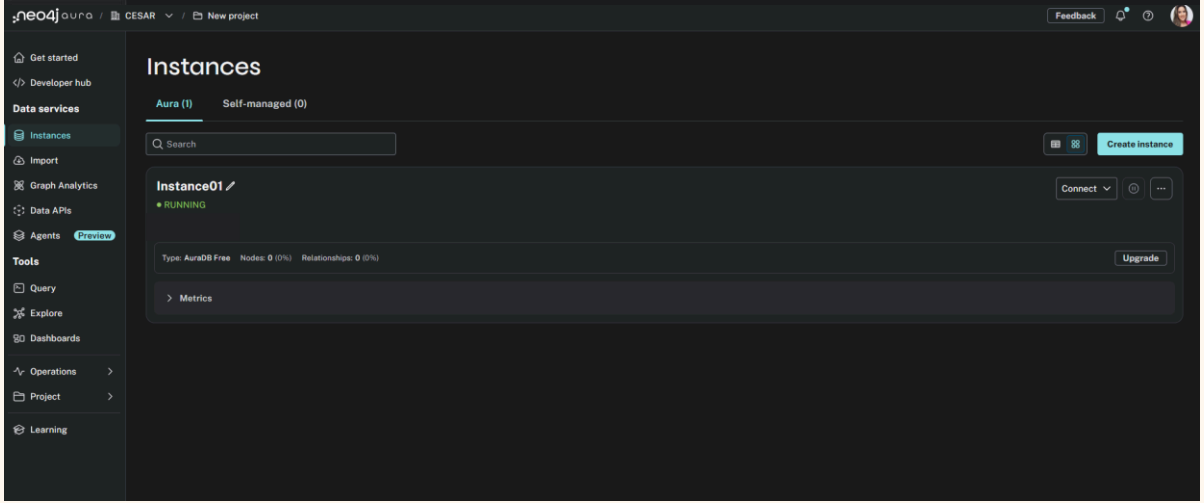
Criando uma instância do Banco AuraDB



Salve a senha gerada para o usuário neo4j

Criando uma instância do Banco AuraDB

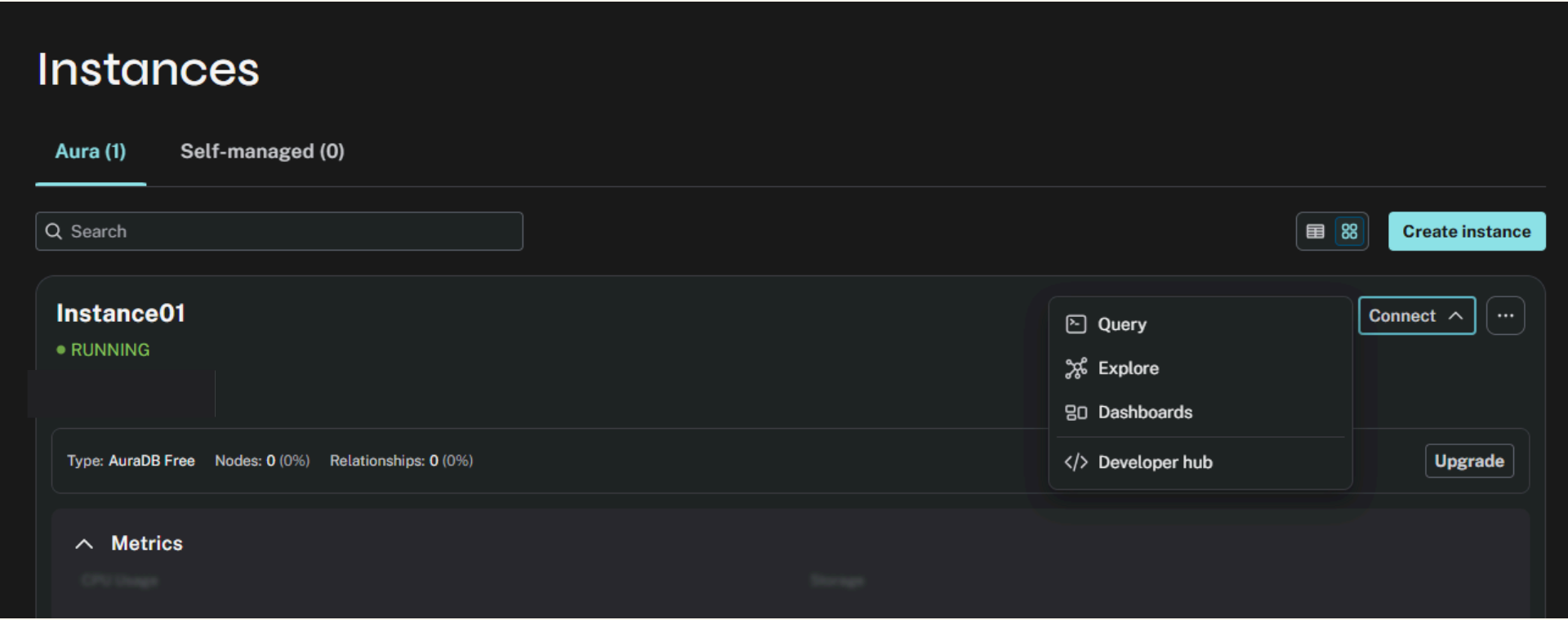




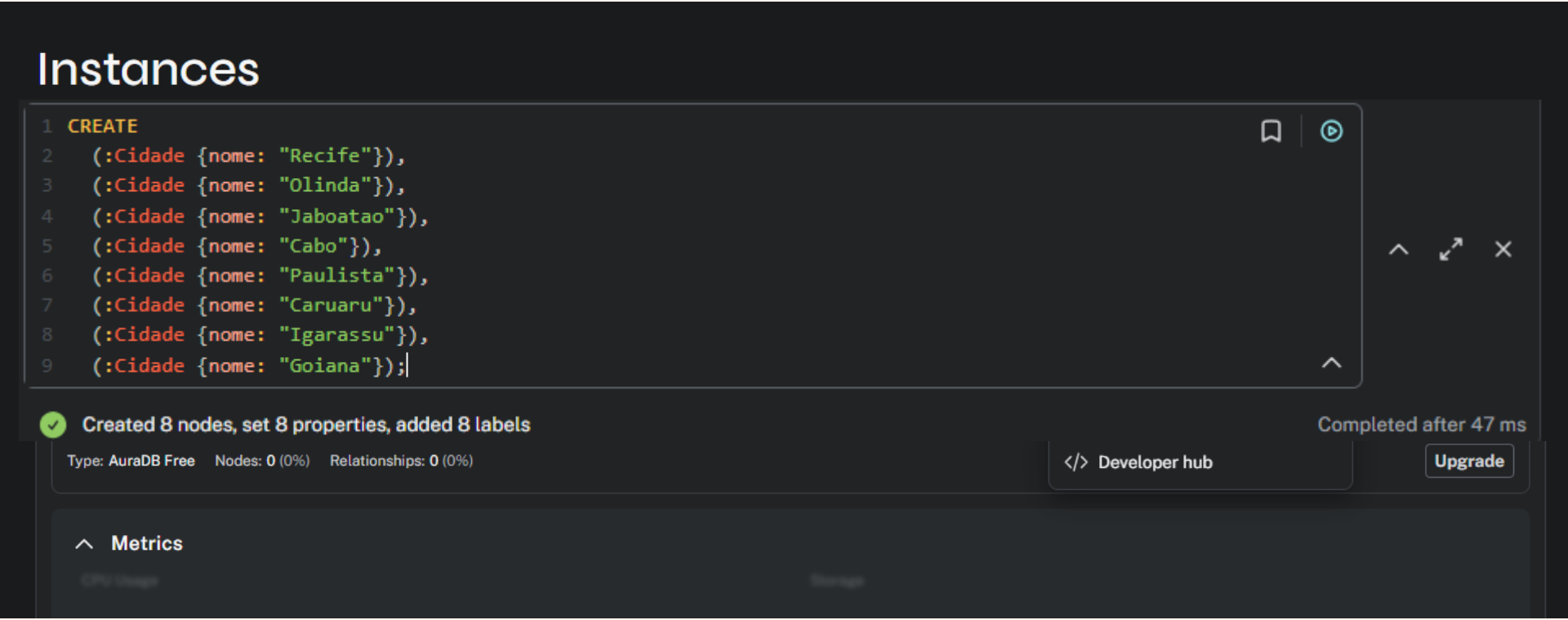
Aguarde o banco ser criado
E disponibilizado online.
“Running” com a cor verde

Após o banco estar running
Clicar em Importar para
Criar um banco de dados (grafo)
A partir de arquivos .csv.

Criando uma instância do Banco AuraDB



Criando uma instância do Banco AuraDB



Dicas para usar o AuraDB Free

C.E.S.A.R school

1

Guarde a senha

Ela aparece uma única vez, na criação da instância. Baixe o arquivo de credenciais.

2

Instância pausada?

Sem uso por 3 dias, o AuraDB Free pausa sozinho. Clique em Resume: os dados continuam lá.

3

Query x Explore

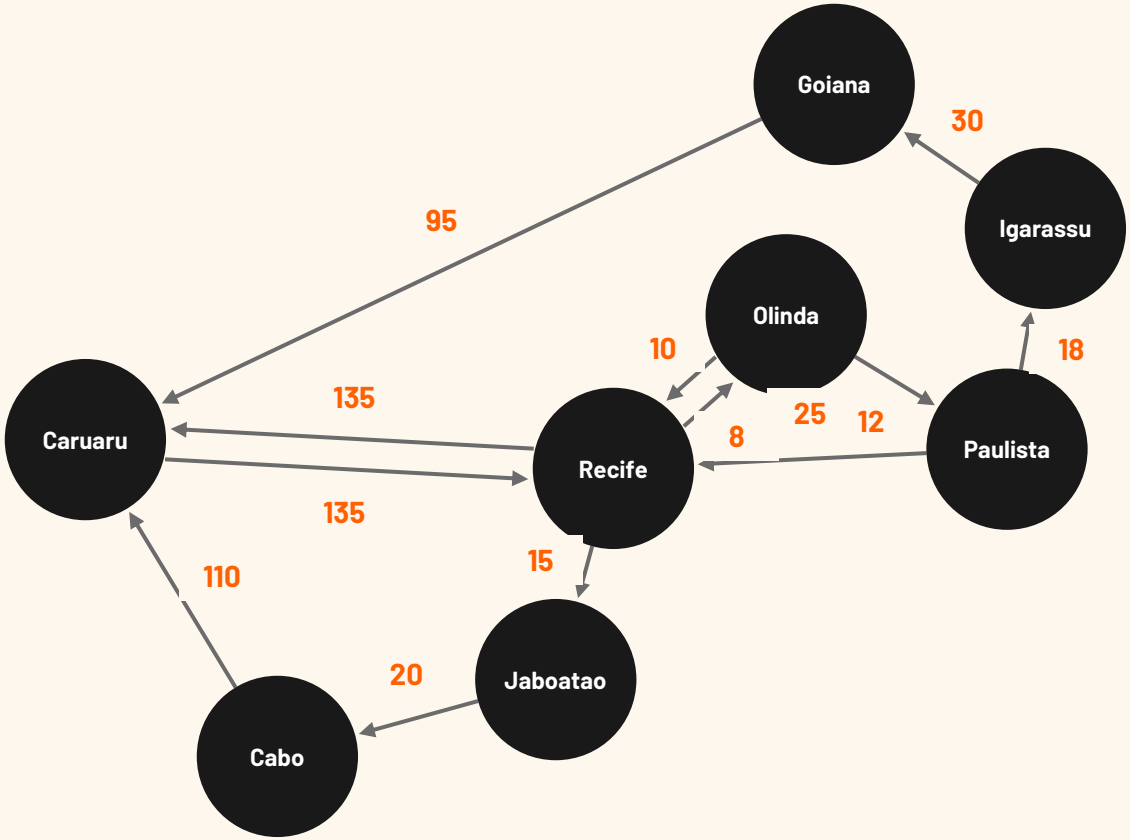
Em Query você executa comandos Cypher. Em Explore você navega visualmente pelo grafo.

4

Salve seus scripts

Guarde os comandos em um arquivo de texto: eles vão para o relatório do projeto.

O grafo que vamos construir



8 nós com o rótulo :Cidade

12 relacionamentos do tipo :ROTA

Grafo orientado e ponderado

O peso de cada aresta é a propriedade distancia, em km.

Distâncias aproximadas, para fins didáticos.

Criar os nós do grafo

```
CREATE
  (:Cidade {nome: "Recife"}),
  (:Cidade {nome: "Olinda"}),
  (:Cidade {nome: "Jaboatao"}),
  (:Cidade {nome: "Cabo"}),
  (:Cidade {nome: "Paulista"}),
  (:Cidade {nome: "Caruaru"}),
  (:Cidade {nome: "Igarassu"}),
  (:Cidade {nome: "Goiana"});
```

Created 8 nodes, set 8 properties, added 8 labels

Completed after 47 ms

```
CREATE
  (:Cidade {nome: "Recife"}),
  (:Cidade {nome: "Olinda"}),
  (:Cidade {nome: "Jaboatao"}),
  (:Cidade {nome: "Cabo"}),
  (:Cidade {nome: "Paulista"}),
  (:Cidade {nome: "Caruaru"}),
  (:Cidade {nome: "Igarassu"}),
  (:Cidade {nome: "Goiana"});
```

O que conferir

A mensagem deve ser: Created 8 nodes, set 8 properties, added 8 labels.

Se aparecer 16, o comando rodou duas vezes.

Entendendo o CREATE

```
CREATE
  (:Cidade {nome: "Recife"}),
  (:Cidade {nome: "Olinda"});
```

- `( )` delimitam um nó
- `:Cidade` é o rótulo, que agrupa nós do mesmo tipo
- `{nome: "Recife"}` é a propriedade do nó
- `,` separa vários nós; `;` encerra o comando

Cuidado: CREATE sempre cria

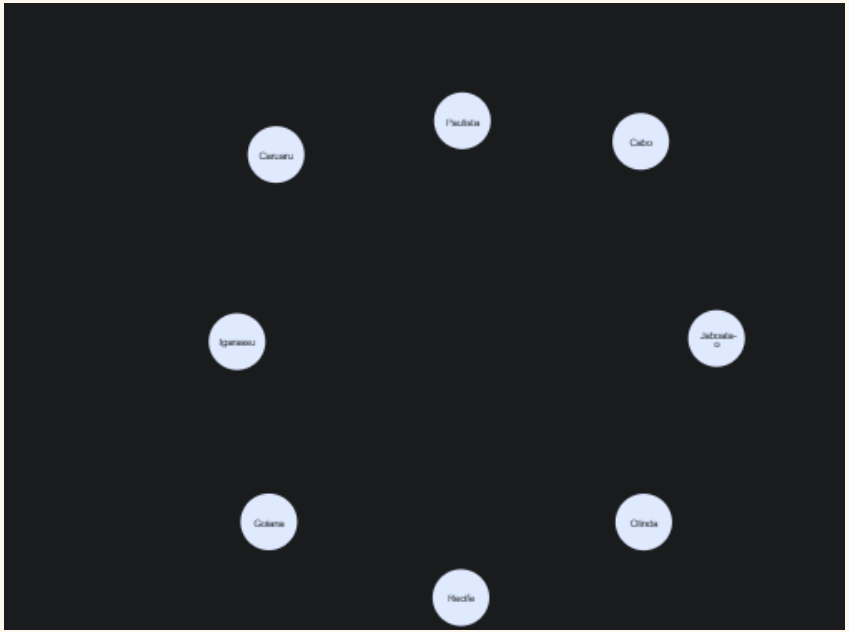
Rodar o mesmo script duas vezes gera cidades duplicadas. Para recomeçar do zero, apague tudo:

```
MATCH (n) DETACH DELETE n;
```

Ou use MERGE, que só cria o que ainda não existe (veja mais adiante).

Visualizar os nós criados

```
1 MATCH (c:Cidade)
2 RETURN c;
```



Criar as rotas entre as cidades

```
MATCH (recife:Cidade {nome: "Recife"})
MATCH (olinda:Cidade {nome: "Olinda"})
MATCH (jaboatao:Cidade {nome: "Jaboatao"})
MATCH (cabo:Cidade {nome: "Cabo"})
MATCH (paulista:Cidade {nome: "Paulista"})
MATCH (caruaru:Cidade {nome: "Caruaru"})
MATCH (igarassu:Cidade {nome: "Igarassu"})
MATCH (goiana:Cidade {nome: "Goiana"})
CREATE
  (recife)-[:ROTA {distancia: 8}]->(olinda),
  (recife)-[:ROTA {distancia: 15}]->(jaboatao),
  (recife)-[:ROTA {distancia: 135}]->(caruaru),
  (olinda)-[:ROTA {distancia: 12}]->(paulista),
  (paulista)-[:ROTA {distancia: 18}]->(igarassu),
  (igarassu)-[:ROTA {distancia: 30}]->(goiana),
  (jaboatao)-[:ROTA {distancia: 20}]->(cabo),
  (cabo)-[:ROTA {distancia: 110}]->(caruaru),
  (goiana)-[:ROTA {distancia: 95}]->(caruaru),
  (paulista)-[:ROTA {distancia: 25}]->(recife),
  (olinda)-[:ROTA {distancia: 10}]->(recife),
  (caruaru)-[:ROTA {distancia: 135}]->(recife);
```

12 arestas orientadas

Cada MATCH guarda uma cidade em uma variável; o CREATE liga as variáveis. A propriedade distancia (km) é o peso da aresta.


```
MATCH (recife:Cidade {nome: "Recife"})
MATCH (olinda:Cidade {nome: "Olinda"})
MATCH (jaboatao:Cidade {nome: "Jaboatao"})
MATCH (cabo:Cidade {nome: "Cabo"})
MATCH (paulista:Cidade {nome: "Paulista"})
MATCH (caruaru:Cidade {nome: "Caruaru"})
MATCH (igarassu:Cidade {nome: "Igarassu"})
MATCH (goiana:Cidade {nome: "Goiana"})
CREATE
(recife)-[:ROTA {distancia: 8}]->(olinda),
(recife)-[:ROTA {distancia: 15}]->(jaboatao),
(recife)-[:ROTA {distancia: 135}]->(caruaru),
(olinda)-[:ROTA {distancia: 12}]->(paulista),
(paulista)-[:ROTA {distancia: 18}]->(igarassu),
(igarassu)-[:ROTA {distancia: 30}]->(goiana),
(jaboatao)-[:ROTA {distancia: 20}]->(cabo),
(cabo)-[:ROTA {distancia: 110}]->(caruaru),
(goiana)-[:ROTA {distancia: 95}]->(caruaru),
(paulista)-[:ROTA {distancia: 25}]->(recife),
(olinda)-[:ROTA {distancia: 10}]->(recife),
(caruaru)-[:ROTA {distancia: 135}]->(recife);
```

Por que MATCH antes do CREATE?

C.E.S.A.R school

Sem MATCH: cria cidades novas

```
CREATE (:Cidade {nome: "Recife"})
  -[:ROTA {distancia: 8}]->
  (:Cidade {nome: "Olinda"});
```

Resultado: mais 2 nós Recife e Olinda, duplicados e desconectados dos originais.

Com MATCH: liga nós existentes

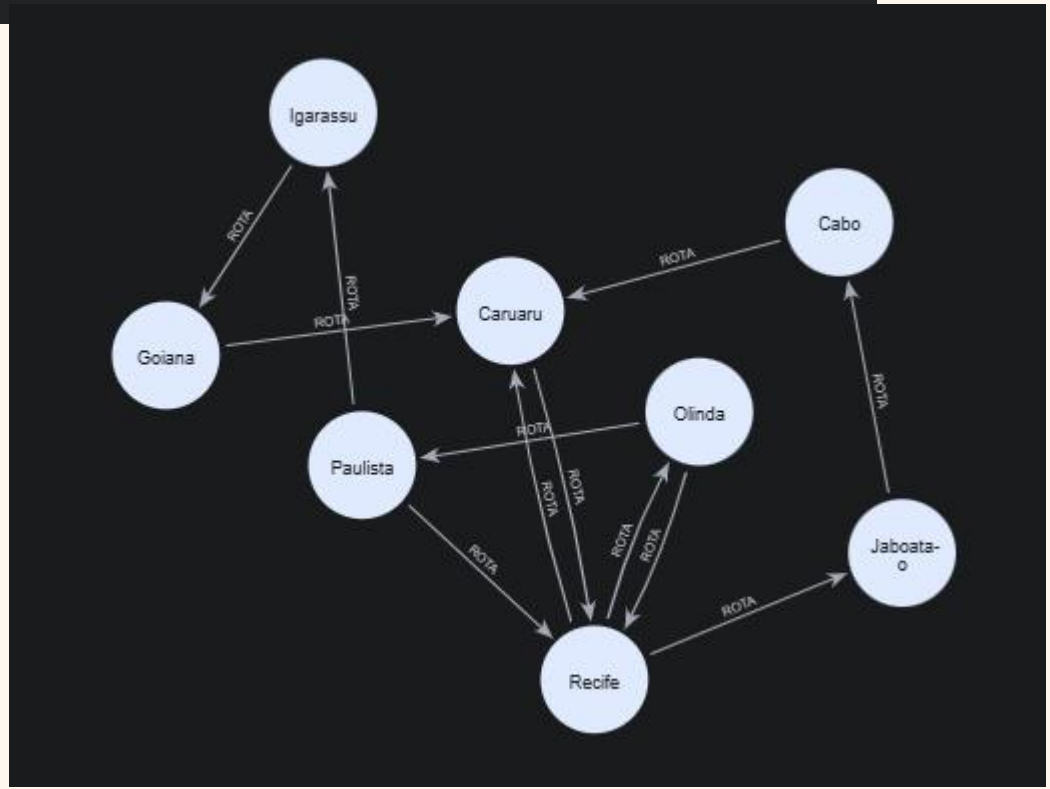
```
MATCH (a:Cidade {nome: "Recife"}),
      (b:Cidade {nome: "Olinda"})
CREATE (a)-[:ROTA {distancia: 8}]->(b);
```

MATCH encontra os nós e guarda em variáveis; CREATE usa as variáveis para criar a aresta.

A direção importa. Recife → Olinda (8 km) e Olinda → Recife (10 km) são dois relacionamentos diferentes: em um dígrafo, $(u, v) \neq (v, u)$.

Visualizar o grafo completo

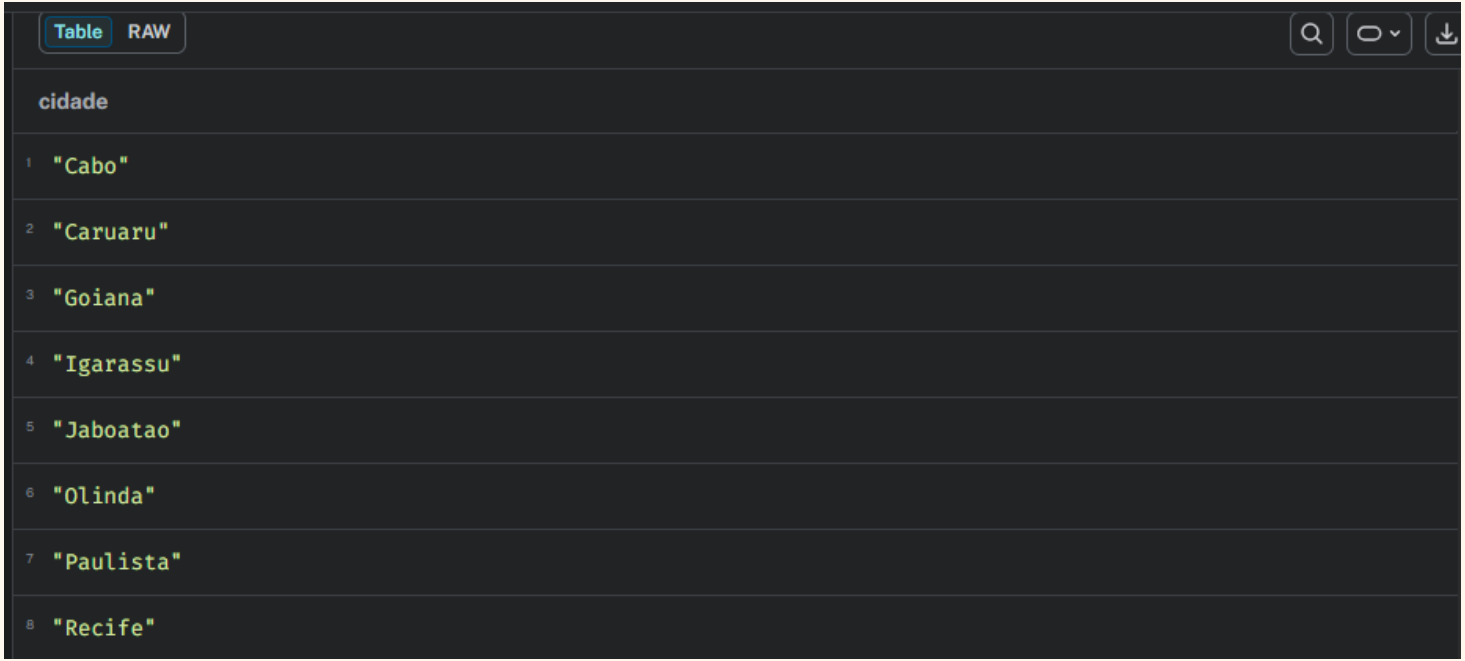
```
1 MATCH (a)-[r:ROTA]->(b)
2 RETURN a, r, b;
```



Clique em um nó ou em uma aresta para ver suas propriedades.

Listar todas as cidades

```
MATCH (c:Cidade)
RETURN c.nome AS cidade
ORDER BY cidade;
```



Listar todas as rotas

```
MATCH (origem:Cidade)-[r:ROTA]->(destino:Cidade)
RETURN origem.nome AS origem, destino.nome AS destino, r.distancia AS distancia
ORDER BY origem, destino;
```

	origem	destino	distancia
1	"Cabo"	"Caruaru"	110
2	"Caruaru"	"Recife"	135
3	"Goiana"	"Caruaru"	95
4	"Igarassu"	"Goiana"	30
5	"Jaboatao"	"Cabo"	20
6	"Olinda"	"Paulista"	12
7	"Olinda"	"Recife"	10

Consultar as rotas que saem de Recife

```
MATCH (origem:Cidade {nome: "Recife"})-[r:ROTA]->(destino:Cidade)
RETURN destino.nome AS destino, r.distancia AS distancia
ORDER BY distancia;
```

	destino	distancia
1	"Olinda"	8
2	"Jaboatao"	15
3	"Caruaru"	135

Isto é o grau de saída de Recife: 3 arestas partem dele. A seguir, o grau de todas as cidades.

Grau de saída e de entrada

```
MATCH (c:Cidade)
RETURN c.nome AS cidade,
  COUNT { (c)-[:ROTA]->() } AS saida,
  COUNT { (c)<-[:ROTA]-() } AS entrada
ORDER BY saida DESC, entrada DESC, cidade;
```

Grau de saída e de entrada

Em um dígrafo, a soma dos graus de saída é igual à soma dos graus de entrada, que é igual ao número de arestas: 12.

Recife e Caruaru recebem mais rotas: 3 cada.

cidade	saída	entrada
Recife	3	3
Olinda	2	1
Paulista	2	1
Caruaru	1	3
Cabo	1	1
Goiana	1	1
Igarassu	1	1
Jaboatao	1	1

Encontrar caminhos de Recife até Caruaru

```
MATCH p = (:Cidade {nome: "Recife"})-[:ROTA*1..4]->(:Cidade {nome: "Caruaru"})
RETURN p;
```



Caminhos de tamanho variável

```
MATCH p = (a:Cidade {nome: "Recife"})-[:ROTA*1..4]->(b:Cidade {nome: "Caruaru"})
```

`p =`
guarda o caminho inteiro em uma variável

`*1..4`
aceita caminhos com 1 a 4 arestas

`nodes(p)`
lista de vértices do caminho

`relationships(p)`
lista de arestas do caminho

`length(p)`
número de arestas (comprimento)

`reduce(...)`
acumula valores, como a soma das distâncias

Sempre limite o tamanho. Com [:ROTA*] sem limite, o número de caminhos cresce exponencialmente em grafos grandes.

Mostrar os caminhos como lista de cidades

```
MATCH p = (:Cidade {nome: "Recife"})-[:ROTA*1..4]->(:Cidade {nome: "Caruaru"})
RETURN
  [cidade IN nodes(p) | cidade.nome] AS caminho,
  length(p) AS quantidade_de_arestas;
```

	caminho	quantidade_de_arestas
1	["Recife", "Caruaru"]	1
2	["Recife", "Olinda", "Recife", "Caruaru"]	3
3	["Recife", "Olinda", "Paulista", "Recife", "Caruaru"]	4
4	["Recife", "Jaboatao", "Cabo", "Caruaru"]	3

Calcular a distância total de cada caminho

```
MATCH p = (:Cidade {nome: "Recife"})-[:ROTA*1..4]->(:Cidade {nome: "Caruaru"})
RETURN
  [cidade IN nodes(p) | cidade.nome] AS caminho,
  length(p) AS quantidade_de_arestas,
  reduce(total = 0, rota IN relationships(p) | total + rota.distancia) AS distancia_total
ORDER BY distancia_total;
```

	caminho	quantidade_de_arestas	distancia_total
1	["Recife", "Caruaru"]	1	135
2	["Recife", "Jaboatao", "Cabo", "Caruaru"]	3	145
3	["Recife", "Olinda", "Recife", "Caruaru"]	3	153
4	["Recife", "Olinda", "Paulista", "Recife", "Caruaru"]	4	180

Trilha ou caminho simples?

No slide anterior apareceram [Recife, Olinda, Recife, Caruaru] e [Recife, Olinda, Paulista, Recife, Caruaru]. O Cypher não repete arestas, mas pode repetir vértices: ele devolve trilhas.

```
MATCH p = (a:Cidade {nome: "Recife"})-[:ROTA*1..4]->(b:Cidade {nome: "Caruaru"})
WHERE all(n IN nodes(p) WHERE single(m IN nodes(p) WHERE m = n))
RETURN [c IN nodes(p) | c.nome] AS caminho, length(p) AS arestas;
```

caminho	arestas
["Recife", "Caruaru"]	1
["Recife", "Jaboatao", "Cabo", "Caruaru"]	3

Leitura do WHERE

Para todo vértice n do caminho, existe um único m igual a ele: nenhum vértice se repete.

Encontrar o menor caminho em número de arestas

```
MATCH p = shortestPath(
  (:Cidade {nome: "Recife"})-[:ROTA*]->(:Cidade {nome: "Caruaru"})
)
RETURN
  [cidade IN nodes(p) | cidade.nome] AS caminho,
  length(p) AS quantidade_de_arestas;
```

caminho	quantidade_de_arestas
["Recife", "Caruaru"]	1

shortestPath conta arestas, não quilômetros. Aqui o resultado coincide com o menor caminho em km (135), mas nem sempre: veja Paulista → Caruaru a seguir.

Menos arestas ≠ menor distância

C.E.S.A.R school

De Paulista até Caruaru:

Menos arestas (shortestPath)

```
MATCH p = shortestPath(
  (:Cidade {nome: "Paulista"})
 -[:ROTA*]->
  (:Cidade {nome: "Caruaru"}))
RETURN [c IN nodes(p) | c.nome];
```

Paulista → Recife → Caruaru

2 arestas · 160 km

Menor distância (soma dos pesos)

```
MATCH p = (:Cidade {nome: "Paulista"})
 -[:ROTA*1..4]->(:Cidade {nome: "Caruaru"})
WITH p, reduce(t = 0, r IN relationships(p)
  | t + r.distancia) AS km
RETURN [c IN nodes(p) | c.nome], km
ORDER BY km LIMIT 1;
```

Paulista → Igarassu → Goiana → Caruaru

3 arestas · 143 km

shortestPath é uma BFS: ignora os pesos. Minimizar a soma dos pesos é o problema do Dijkstra (ou Bellman-Ford, com pesos negativos).

Alterando o grafo

MERGE: cria só se não existir

```
MERGE (v:Cidade {nome: "Vitoria"});
```

MERGE em relacionamento

```
MATCH (r:Cidade {nome: "Recife"}),
      (v:Cidade {nome: "Vitoria"})
MERGE (r)-[:ROTA {distancia: 50}]->(v);
```

SET: cria ou altera propriedade

```
MATCH (r:Cidade {nome: "Recife"})
SET r.capital = true
RETURN r;
```

DETACH DELETE: apaga nó e arestas

```
MATCH (v:Cidade {nome: "Vitoria"})
DETACH DELETE v;
```

Evite duplicatas no projeto: `CREATE CONSTRAINT cidade_nome IF NOT EXISTS FOR (c:Cidade) REQUIRE c.nome IS UNIQUE;`

Teoria dos Grafos no Cypher

Conceito	Em Cypher	No nosso grafo
Grau de saída	`COUNT { (c)-[:ROTA]->() }`	Recife tem grau de saída 3
Caminho	`(a)-[:ROTA*1..4]->(b)`	Recife → Jaboatao → Cabo → Caruaru
Menor caminho (arestas): BFS	`shortestPath((a)-[:ROTA*]->(b))`	Paulista → Recife → Caruaru
Menor caminho (pesos): Dijkstra	`reduce + ORDER BY + LIMIT 1`	Paulista → Igarassu → Goiana → Caruaru
Ciclo	`(a)-[:ROTA*2..5]->(a)`	Recife → Olinda → Paulista → Recife
Alcançabilidade	`(a)-[:ROTA*]->(b)`	Toda cidade alcança todas: fortemente conexo

Exercícios em sala

1. Liste as rotas com menos de 20 km, da menor para a maior.
2. Qual é a soma de todas as distâncias? E a média por rota?
3. Quais cidades podem ser alcançadas a partir de Goiana?
4. Qual cidade recebe mais rotas (maior grau de entrada)?
5. Qual o caminho com menos arestas de Caruaru até Goiana? Quantos km ele tem?

Gabarito: exercícios 1 a 3

1

```
MATCH (a:Cidade)-[r:ROTA]->(b:Cidade)
WHERE r.distancia < 20
RETURN a.nome, b.nome, r.distancia AS km
ORDER BY km;
```

5 rotas: Recife→Olinda 8 · Olinda→Recife 10 · Olinda→Paulista 12 · Recife→Jaboatao 15 · Paulista→Igarassu 18

2

```
MATCH ()-[r:ROTA]->()
RETURN sum(r.distancia) AS total,
  round(avg(r.distancia), 1) AS media;
```

Total de 613 km; média de 51,1 km por rota

3

```
MATCH (:Cidade {nome: "Goiana"})-[:ROTA*]->(d:Cidade)
RETURN DISTINCT d.nome AS cidade
ORDER BY cidade;
```

Todas as 8 cidades, inclusive a própria Goiana (por um ciclo)

Gabarito: exercícios 4 e 5

4

```cypher
MATCH (c:Cidade)<-[:ROTA]-()
RETURN c.nome AS cidade,
  count(*) AS entrada
ORDER BY entrada DESC;
```

Empate: Recife e Caruaru, com 3 rotas cada. Com LIMIT 1, o empate ficaria escondido!

5

```cypher
MATCH p = shortestPath(
  (:Cidade {nome: "Caruaru"})
  -[:ROTA*]->(:Cidade {nome: "Goiana"}))
RETURN [c IN nodes(p) | c.nome] AS caminho,
  length(p) AS arestas,
  reduce(t = 0, r IN relationships(p)
    | t + r.distancia) AS km;
```

Caruaru → Recife → Olinda → Paulista → Igarassu → Goiana

5 arestas · 203 km

Fechando a disciplina

Os algoritmos que vocês implementaram estão por trás dessas ferramentas.

Fundamentos
Vértices, arestas, graus e representações

Buscas
BFS, DFS e A*

Caminhos mínimos
Dijkstra e Bellman-Ford

Árvores e percursos
Árvore geradora mínima, Euler e Hamilton

Fluxo em redes
Capacidades, fluxo máximo e corte mínimo

Hoje: bancos de grafos
Os mesmos grafos, agora em um banco

Dúvidas?



Laura Alves Pacifico
laps@cesar.school
Slack: Laura Pacifico